SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OPERATING SYSTEMS **COURSE CODE: CSA0416**
NAME:PREETHI R **REG NO:192511352**

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 40%

QUESTIONS: 10 **Total Marks:100**

Annexure A

Analytical Problem Solving

Code of Conduct:

I __PREETHI R/12511352__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Question 1: CPU Scheduling – Waiting Time & Turnaround Time Analysis

A set of processes arrive at time 0 with the following burst times:

Process	Burst Time (ms)
P1	10
P2	4
P3	6
P4	2

a. Calculate Waiting Time and Turnaround Time for each process using SJF (Non-preemptive).
b. Compute the average waiting time and average turnaround time.
c. Interpret whether SJF improves CPU utilization compared to FCFS for this workload.

(10 Marks)

Question 2: Round Robin Scheduling – Time Quantum Impact

Consider the following processes:

Process	Burst Time (ms)
P1	20
P2	5
P3	13
P4	8

Let the time quantum = 4 ms.

a. Construct the Gantt chart for Round Robin.
b. Calculate Waiting Time and Turnaround Time of each process.
c. Analyze how changing the quantum to 10 ms would affect system performance (CPU utilization & context switching).

(10 Marks)

Question 3: Deadlock – Safety Check Using Banker's Algorithm

A system has 3 resource types (A, B, C). Total resources:
A = 10, B = 5, C = 7.

The Allocation and Maximum Need matrices are:

Allocation Matrix

Process	A	B	C
P1	0	1	0
P2	2	0	0
P3	3	0	2
P4	2	1	1
P5	0	0	2

Maximum Matrix

Process	A	B	C
P1	7	5	3
P2	3	2	2
P3	9	0	2
P4	4	2	2
P5	5	3	3

a. Calculate the Need Matrix.
b. Determine if the system is in a safe state using Banker’s Algorithm.
c. Provide a valid safe sequence if it exists.

(10 Marks)

Question 4: Deadlock Detection – Resource Allocation Graph (RAG)

A system has the following resource allocations:

- P1 is holding R1 and waiting for R2

- P2 is holding R2 and waiting for R3
- P3 is holding R3 and waiting for R1

a. Draw the Resource Allocation Graph (describe in text).
b. Determine whether the system is in deadlock.
c. Recommend a strategy (prevention OR avoidance OR recovery) to resolve it and justify the method.

(10 Marks)

Question 5: CPU Utilization – Multiprogramming Level Analysis

A system has the following process behavior:

- Each process spends 20% time in CPU and 80% waiting for I/O.
- Degree of multiprogramming = 4 processes

CPU Utilization formula:

$$U = 1 - (p^n)$$

where:

- p = probability a process is waiting for I/O,
- n = number of processes in memory

a. Calculate CPU Utilization.
b. Compute CPU utilization if the degree of multiprogramming increases to 6.
c. Interpret which configuration improves system performance and why.

(10 Marks)

Question 6: Producer–Consumer Problem Using Semaphores

A bounded buffer has a capacity of 6 items. Initially, the buffer is empty. Three producer operations occur followed by two consumer operations.

a. Show the values of the semaphores (empty, full, mutex) after each operation.
b. Explain how semaphores prevent race conditions.
c. What problem occurs if the mutex semaphore is removed? Justify your answer.

(10 Marks)

Question 7: Process Synchronization – Critical Section

Two processes increment a shared variable X initially equal to 100. Each process executes the increment operation 1000 times without synchronization.

a. Explain how a race condition occurs.
b. Determine the expected value of X with proper synchronization.
c. Suggest one hardware-based and one software-based synchronization solution.

(10 Marks)

Question 8: Thread Scheduling and Performance

A program creates 40 user-level threads on a system having 8 CPU cores. Each thread requires 12 ms CPU time and performs negligible I/O.

a. Compare the Many-to-One and One-to-One multithreading models.
b. Estimate how CPU utilisation differs between the two models.
c. Recommend the better threading model with justification.

(10 Marks)

Question 9: Monitor Synchronization

A monitor controls access to a shared printer used by multiple processes.

a. Explain how monitors ensure mutual exclusion.
b. Describe the role of condition variables wait() and signal().
c. Explain what may happen if signal() is omitted after a print job completes.

(10 Marks)

Question 10: Deadlock Prevention vs Avoidance

A system has four processes competing for two reusable resources.

a. Explain how deadlock prevention can eliminate one of the four necessary conditions for deadlock.
b. Using a suitable example, explain how deadlock avoidance differs from prevention.
c. Compare prevention and avoidance in terms of resource utilisation and system performance.

(10 Marks)

QUESTION 1: CPU SCHEDULING – WAITING TIME AND TURNAROUND TIME ANALYSIS (10 MARKS)

(a) Step 1: Arrange processes using SJF

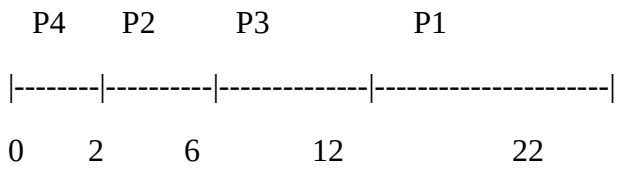
Given: All processes arrive at time 0.

Process	Burst Time (ms)	Arrival Time (ms)
P1	10	0
P2	4	0
P3	6	0
P4	2	0

Arrange the burst times from shortest to longest: P4 (2), P2 (4), P3 (6), P1 (10).

Execution order: P4 → P2 → P3 → P1.

Gantt Chart:



(a) Step 2: Calculate waiting and turnaround times

Waiting Time (WT) = Start Time − Arrival Time

Turnaround Time (TAT) = Completion Time − Arrival Time

P4: WT = 0 − 0 = 0 ms; TAT = 2 − 0 = 2 ms.

P2: WT = 2 − 0 = 2 ms; TAT = 6 − 0 = 6 ms.

P3: WT = 6 − 0 = 6 ms; TAT = 12 − 0 = 12 ms.

P1: WT = 12 − 0 = 12 ms; TAT = 22 − 0 = 22 ms.

Process	Burst (ms)	Start	Completion	WT (ms)	TAT (ms)
P4	2	0	2	0	2
P2	4	2	6	2	6
P3	6	6	12	6	12
P1	10	12	22	12	22

(b) Step 3: Calculate averages

Average Waiting Time:

$$\mathbf{AWT = (0 + 2 + 6 + 12) / 4 = 20 / 4 = 5\ ms}$$

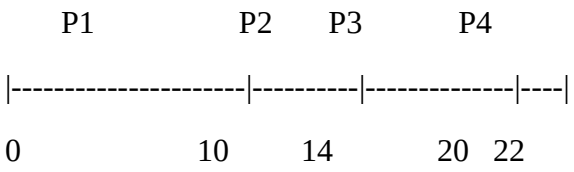
Average Turnaround Time:

$$\mathbf{ATAT = (2 + 6 + 12 + 22) / 4 = 42 / 4 = 10.5\ ms}$$

(c) Step 4: Compare with FCFS

FCFS execution order: P1 → P2 → P3 → P4.

Gantt Chart:



Process	**Waiting Time (ms)**	**Turnaround Time (ms)**
P1	0	10
P2	10	14
P3	14	20
P4	20	22
Average	11	16.5

FCFS average WT = (0 + 10 + 14 + 20) / 4 = 11 ms.

FCFS average TAT = (10 + 14 + 20 + 22) / 4 = 16.5 ms.

Interpretation: SJF reduces average waiting time from 11 ms to 5 ms and average turnaround time from 16.5 ms to 10.5 ms. Both methods provide 100% ideal CPU utilization because the CPU remains busy for 22 ms, assuming no scheduling overhead.

Result: SJF gives lower average waiting and turnaround times than FCFS for the given processes.

QUESTION 2: ROUND ROBIN SCHEDULING – TIME QUANTUM IMPACT (10 MARKS)

(a) Step 1: Gantt chart for quantum = 4 ms

Given: P1 = 20 ms, P2 = 5 ms, P3 = 13 ms, P4 = 8 ms. All arrive at time 0 in the order P1, P2, P3, P4.

Each process runs for up to 4 ms. If it is unfinished, it returns to the end of the ready queue.

Gantt Chart:

P1 | P2 | P3 | P4 | P1 | P2 | P3 | P4 | P1 | P3 | P1 | P3 | P1

0 4 8 12 16 20 21 25 29 33 37 41 42 46

Remaining burst time calculations:

P1: 20 → 16 → 12 → 8 → 4 → 0; completion time = 46 ms.

P2: 5 → 1 → 0; completion time = 21 ms.

P3: 13 → 9 → 5 → 1 → 0; completion time = 42 ms.

P4: 8 → 4 → 0; completion time = 29 ms.

(b) Step 2: Calculate waiting and turnaround times

TAT = Completion Time − Arrival Time; WT = TAT − Burst Time.

P1: TAT = 46 − 0 = 46 ms; WT = 46 − 20 = 26 ms.

P2: TAT = 21 − 0 = 21 ms; WT = 21 − 5 = 16 ms.

P3: TAT = 42 − 0 = 42 ms; WT = 42 − 13 = 29 ms.

P4: TAT = 29 − 0 = 29 ms; WT = 29 − 8 = 21 ms.

Process	**Burst (ms)**	**Completion (ms)**	**TAT (ms)**	**WT (ms)**
P1	20	46	46	26
P2	5	21	21	16
P3	13	42	42	29
P4	8	29	29	21
Average	—	—	34.5	23

Average WT = (26 + 16 + 29 + 21) / 4 = 92 / 4 = 23 ms

Average TAT = (46 + 21 + 42 + 29) / 4 = 138 / 4 = 34.5 ms

(c) Step 3: Change the quantum to 10 ms

Gantt Chart:

P1 | P2 | P3 | P4 | P1 | P3

0 10 15 25 33 43 46

Completion times: P1 = 43 ms, P2 = 15 ms, P3 = 46 ms, P4 = 33 ms.

Process	Burst (ms)	Completion (ms)	TAT (ms)	WT (ms)
P1	20	43	43	23
P2	5	15	15	10
P3	13	46	46	33
P4	8	33	33	25
Average	—	—	34.25	22.75

Average WT = (23 + 10 + 33 + 25) / 4 = 22.75 ms

Average TAT = (43 + 15 + 46 + 33) / 4 = 34.25 ms

Interpretation: A 10 ms quantum reduces preemptions and context switches, lowering overhead, but can reduce responsiveness. Ideal CPU utilization remains 100% for both quantum values because the CPU stays busy for the 46 ms total work, ignoring overhead.

Result: For quantum 4 ms, average WT = 23 ms and average TAT = 34.5 ms. For quantum 10 ms, average WT = 22.75 ms and average TAT = 34.25 ms.

QUESTION 3: DEADLOCK – SAFETY CHECK USING BANKER'S ALGORITHM (10 MARKS)

(a) Step 1: Calculate the Need Matrix

Formula: Need = Maximum − Allocation.

Process	Allocation (A,B,C)	Maximum (A,B,C)	Need (A,B,C)
P1	(0,1,0)	(7,5,3)	(7,4,3)
P2	(2,0,0)	(3,2,2)	(1,2,2)
P3	(3,0,2)	(9,0,2)	(6,0,0)

P4	(2,1,1)	(4,2,2)	(2,1,1)
P5	(0,0,2)	(5,3,3)	(5,3,1)

P1: Need = (7−0, 5−1, 3−0) = (7,4,3).

P2: Need = (3−2, 2−0, 2−0) = (1,2,2).

P3: Need = (9−3, 0−0, 2−2) = (6,0,0).

P4: Need = (4−2, 2−1, 2−1) = (2,1,1).

P5: Need = (5−0, 3−0, 3−2) = (5,3,1).

(b) Step 2: Calculate available resources

Allocated A = 0 + 2 + 3 + 2 + 0 = 7.

Allocated B = 1 + 0 + 0 + 1 + 0 = 2.

Allocated C = 0 + 0 + 2 + 1 + 2 = 5.

Available = Total − Allocated = (10−7, 5−2, 7−5) = (3,3,2)

(c) Step 3: Apply the safety algorithm

Start with Work = Available = (3,3,2). Choose a process whose Need is less than or equal to Work. When it finishes, add its Allocation to Work.

Step	Process	Need	Work Before	Allocation Returned	Work After
1	P2	(1,2,2)	(3,3,2)	(2,0,0)	(5,3,2)
2	P4	(2,1,1)	(5,3,2)	(2,1,1)	(7,4,3)
3	P1	(7,4,3)	(7,4,3)	(0,1,0)	(7,5,3)
4	P3	(6,0,0)	(7,5,3)	(3,0,2)	(10,5,5)
5	P5	(5,3,1)	(10,5,5)	(0,0,2)	(10,5,7)

Safe sequence: P2 → P4 → P1 → P3 → P5

Interpretation: At every step, the selected process has enough available resources to finish. All five processes can complete and release their allocated resources.

Result: The system is in a safe state because a valid safe sequence exists.

QUESTION 4: DEADLOCK DETECTION – RESOURCE ALLOCATION GRAPH (10 MARKS)

(a) Step 1: Write allocation and request edges

R1 → P1 (R1 is allocated to P1).

P1 → R2 (P1 requests R2).

R2 → P2 (R2 is allocated to P2).

P2 → R3 (P2 requests R3).

R3 → P3 (R3 is allocated to P3).

P3 → R1 (P3 requests R1).

Text representation of the cycle:

P1 → R2 → P2 → R3 → P3 → R1 → P1

(b) Step 2: Determine whether deadlock exists

Assuming each resource has one instance, the cycle confirms deadlock. P1 waits for R2 held by P2; P2 waits for R3 held by P3; P3 waits for R1 held by P1. None can proceed to release the resource needed by the next process.

(c) Step 3: Recommend a recovery strategy

Terminate one process or preempt a resource if safe. For example, terminating P3 releases R3. P2 can then obtain R3 and finish, release R2, and allow P1 to proceed. The operating system should choose a victim based on priority, work completed, resources held and restart cost.

Result: A circular wait exists. With single-instance resources, the system is deadlocked; recovery breaks the cycle.

QUESTION 5: CPU UTILIZATION – MULTIPROGRAMMING LEVEL ANALYSIS (10 MARKS)

(a) Step 1: Calculate utilization for 4 processes

Given: $p = 0.8$ and $n = 4$.

Formula: CPU Utilization $= 1 - p^n$.

$$\mathbf{U = 1 - (0.8)^4 = 1 - 0.4096 = 0.5904 = 59.04\%}$$

(b) Step 2: Calculate utilization for 6 processes

Given: $p = 0.8$ and $n = 6$.

$$\mathbf{U = 1 - (0.8)^6 = 1 - 0.262144 = 0.737856 \approx 73.79\%}$$

Degree of Multiprogramming	Calculation	CPU Utilization
4 processes	$1 - (0.8)^4$	59.04%
6 processes	$1 - (0.8)^6$	73.79%

(c) Step 3: Interpret the result

Utilization increases by 73.79 − 59.04 = 14.75 percentage points when the number of processes increases from 4 to 6. With more processes, it is less likely that all processes will wait for I/O simultaneously, so the CPU is more likely to have work.

This is an idealized estimate. Excessive multiprogramming may increase memory pressure and scheduling overhead.

Result: Six processes give higher estimated CPU utilization (73.79%) than four processes (59.04%).

QUESTION 6: PRODUCER–CONSUMER PROBLEM USING SEMAPHORES (10 MARKS)

(a) Step 1: Initialize semaphores

Buffer capacity = 6. Initial values: empty = 6, full = 0, mutex = 1.

empty counts available slots, full counts occupied slots, and mutex controls exclusive access to the buffer.

(a) Step 2: Update values after each operation

Stage	Operation	empty	full	mutex
Initial	Buffer is empty	6	0	1
1	Producer 1 inserts an item	5	1	1
2	Producer 2 inserts an item	4	2	1
3	Producer 3 inserts an item	3	3	1
4	Consumer 1 removes an item	4	2	1

5	Consumer 2 removes an item	5	1	1

Producer sequence: wait(empty) → wait(mutex) → insert → signal(mutex) → signal(full).

Consumer sequence: wait(full) → wait(mutex) → remove → signal(mutex) → signal(empty).

(b) Step 3: Explain prevention of race conditions

The empty semaphore prevents producers from inserting when the buffer is full. The full semaphore prevents consumers from removing items when the buffer is empty. The mutex allows only one process to modify the buffer at a time, protecting shared data from simultaneous updates.

(c) Step 4: Effect of removing mutex

Without mutex, two producers could write to the same slot or update the buffer index simultaneously. A consumer could access the buffer while another process modifies it. This may overwrite items, corrupt the buffer count or cause inconsistent data.

Result: Final values after three producer and two consumer operations: empty = 5, full = 1, mutex = 1.

QUESTION 7: PROCESS SYNCHRONIZATION – CRITICAL SECTION (10 MARKS)

(a) Step 1: Explain the race condition

Each increment involves reading X, adding 1, and writing the new value. If both processes read the same old value before either writes, both can write the same result and one increment is lost.

Example: X = 100; P1 reads 100; P2 reads 100; P1 writes 101; P2 writes 101. Two increments were attempted, but X increases only once.

(b) Step 2: Calculate expected value

Each process performs 1,000 increments. Total increments = 1,000 + 1,000 = 2,000.

Final X = Initial X + Total increments = 100 + 2,000 = 2,100

(c) Step 3: Give synchronization solutions

Hardware-based solution: Use an atomic instruction such as Test-and-Set, Compare-and-Swap (CAS), or atomic fetch-and-add.

Software-based solution: Use a mutex lock or semaphore. A process acquires the lock, updates X inside the critical section, and releases the lock afterward.

Result: With proper synchronization, the final value of X is 2100.

QUESTION 8: THREAD SCHEDULING AND PERFORMANCE (10 MARKS)

(a) Step 1: Compare the threading models

Feature	Many-to-One	One-to-One
Mapping	Many user threads map to one kernel thread	Each user thread maps to a kernel thread
Parallel execution	Cannot run in parallel across multiple cores	Can run multiple threads on multiple cores
Blocking	One blocking call may block all user threads	A blocked thread usually does not block all others
Overhead	Lower kernel-thread management overhead	Higher thread creation and management overhead
Multicore use	One execution stream	Can use multiple cores

(b) Step 2: Estimate CPU utilization

Total CPU work = 40 threads × 12 ms = 480 CPU-ms.

Many-to-One: One kernel thread executes at a time. At most 1 of 8 cores is used. Maximum utilization relative to the full machine is 1/8 × 100 = 12.5%, assuming that core is fully busy. Ideal elapsed time = 480 ms.

One-to-One: Up to 8 threads can execute simultaneously. Ideal elapsed time = 480 / 8 = 60 ms. If all 8 cores are busy, the application uses 100% of total CPU capacity.

These are ideal estimates assuming all threads are runnable, no competing workload and negligible scheduling overhead.

(c) Step 3: Recommend a model

One-to-One is recommended because it allows parallel execution across eight cores and reduces elapsed time. It has higher kernel-thread management overhead, but the parallelism is valuable for this CPU-bound workload.

Result: One-to-One is better suited to this system because it can use all eight CPU cores.

QUESTION 9: MONITOR SYNCHRONIZATION (10 MARKS)

(a) Step 1: Explain mutual exclusion

A monitor groups shared data and the procedures that access it. Only one process or thread can execute inside the monitor at a time. For a shared printer, this prevents two processes from changing printer state or starting conflicting jobs simultaneously.

(b) Step 2: Explain wait() and signal()

wait(): If the printer is busy, a process waits on a condition variable. It releases the monitor lock while waiting, allowing another process to enter the monitor.

signal(): When a print job completes and the printer becomes available, the process signals the condition variable so a waiting process can continue.

(c) Step 3: Explain what happens if signal() is omitted

If signal() is omitted after a print job completes, processes waiting for the printer may remain blocked even though the printer is free. This can cause indefinite waiting and prevent efficient printer use.

Result: Monitors provide mutual exclusion, while condition variables coordinate waiting and notification when the printer becomes available.

QUESTION 10: DEADLOCK PREVENTION VS AVOIDANCE (10 MARKS)

(a) Step 1: Explain deadlock prevention

Deadlock requires four conditions: mutual exclusion, hold and wait, no preemption, and circular wait. Prevention makes at least one condition impossible.

Example: Break hold and wait by requiring a process to request all the resources it needs before execution. If all are unavailable, it receives none and waits. This prevents it from holding one resource while waiting for another, but may reduce resource utilization.

(b) Step 2: Explain deadlock avoidance with an example

Avoidance checks each resource request and grants it only if the resulting state remains safe. For example, if P1 and P2 compete for resources R1 and R2, the system can delay a request that would leave no safe completion sequence and grant it when safety can be maintained.

Prevention uses fixed rules to break a necessary deadlock condition. Avoidance makes decisions using current allocations, available resources and declared maximum needs.

(c) Step 3: Compare prevention and avoidance

Aspect	**Prevention**	**Avoidance**
Approach	Breaks at least one necessary condition	Checks whether each allocation leaves a safe state
Information	Can use fixed rules	Needs allocation, available resources and maximum needs

Resource utilization	May be lower due to restrictive rules	Can be higher by granting safe requests
Overhead	Usually lower runtime checking	Higher due to safety checks
Flexibility	Less flexible	More flexible if resource needs are known

Interpretation: Prevention is simpler but may restrict resource use even when a request could be safe. Avoidance permits more flexible allocation, but needs resource-claim information and extra checking.

Result: Prevention eliminates a necessary deadlock condition; avoidance grants requests only when the system remains safe.

RUBRICS FOR CALCULATION: Analytical Problem Solving

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Marks Evaluation:

Question No.	Problem Understandin (20%)	Method Selection(20%)	Solution Process(25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)	Total Marks (100)
Q1						
Q2						
Q3						
Q4						
Q5						
Q6						

Q7						
Q8						
Q9						
Q10						
					Overall Total (100)	

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____